Tampereen yliopisto

COMP.SE.110
SOFTWARE DESIGN
MID-TERM DOCUMENTATION
Event-Planner Dashboard: Weather and Air Quality

Naveed Ali
Affira
Zhiqing Feng
Janika Ranta

Table of Contents

1. PROJECT OVERVIEW 3
2. STRUCTURE AND KEY COMPONENTS 3
3. COMPONENTS AND INTERFACES 4
4. DESIGN PATTERNS AND ARCHITECTURE 5
5. APIS AND DATA 6
6. SAVED PREFERENCES 7
7. HISTORY 8
8. TENTATIVE APPROACHES 8
9. SELF-EVALUATION 9
10. CONCLUSION 11
11. DIVISION OF WORK 11

Event Planner's Dashboard

1. Project Overview

The **Event Planner's Dashboard** is designed to help event planners choose the best dates and locations for outdoor events based on real-time weather forecasts and air quality data. The application fetches data from two APIs: one for weather information and another for air quality metrics, allowing users to make informed decisions for event scheduling. The dashboard provides customizable visualizations for weather conditions and air quality levels. Users can save their preferences for future use and access their historical data to compare trends over time.

User Stories/Use Cases:

- **As a professional event planner**, I want to view detailed weather and air quality forecasts so I can choose the best dates for outdoor events.
- **As an event venue manager**, I want to compare historical data with current weather trends to make informed decisions about venue bookings.
- **As a casual user**, I want to check real-time weather and air quality data for my favorite locations so I can plan outings and activities.

2. Structure and Key Components

The dashboard is divided into three main sections:

1. **Weather Data Section**: Displays data such as temperature, humidity, and wind speed in line graphs.
2. **Air Quality Data Section**: Visualizes air quality metrics (e.g., PM2.5, PM10) using bar charts.
3. **Map and Preferences Section**: Allows users to select locations from a map and save preferences for easy future access.

Main Components:

- **Location and Date Picker**: A dropdown menu for selecting regions and a date picker for specifying date ranges.
- **Weather Visualizations**: Provides a graphical representation of key weather metrics such as temperature, humidity, and wind speed. This data is fetched from an external API and displayed in interactive line charts for better visual interpretation.
- **Air Quality Visualizations**: Offers visualizations of PM2.5 and PM10 levels using bar charts. These indicators help users understand air quality trends and how they might affect events.

- **Map and Preferences Section**: Allows users to select locations on a map and save them for easy future access. Preferences include the date range and data filters for weather and air quality metrics.

3. Components and Interfaces

The project will utilize a **Model-View-Controller (MVC)** architecture to separate data processing, user interaction, and interface rendering. Here are the core components:

1. **Weather API Handler**:

   Handles requests to external weather data providers (e.g., OpenWeatherMap or Finnish Meteorological Institute). It fetches weather details such as temperature, humidity, wind speed, and precipitation levels in JSON format. The handler validates responses and parses them into internal data models.

2. **Air Quality API Handler**:

   Connects to the OpenAQ API to fetch real-time air quality data. It retrieves JSON responses containing air quality metrics like PM2.5 and PM10. The handler is responsible for converting this data into a format usable by the visualization components.

3. **Controller**:

   The controller receives user input from the UI, triggers the appropriate API handlers, and passes the fetched data to be displayed. It coordinates all interactions between the API handlers and the UI components.

4. **User Interface (UI)**:

   Built using JavaFX, the UI is responsible for displaying real-time data visualizations. It allows users to input locations, select dates, save preferences, and view historical weather conditions. The UI components include line graphs for weather data, bar charts for air quality, and a map-based location picker.

Class Diagram:

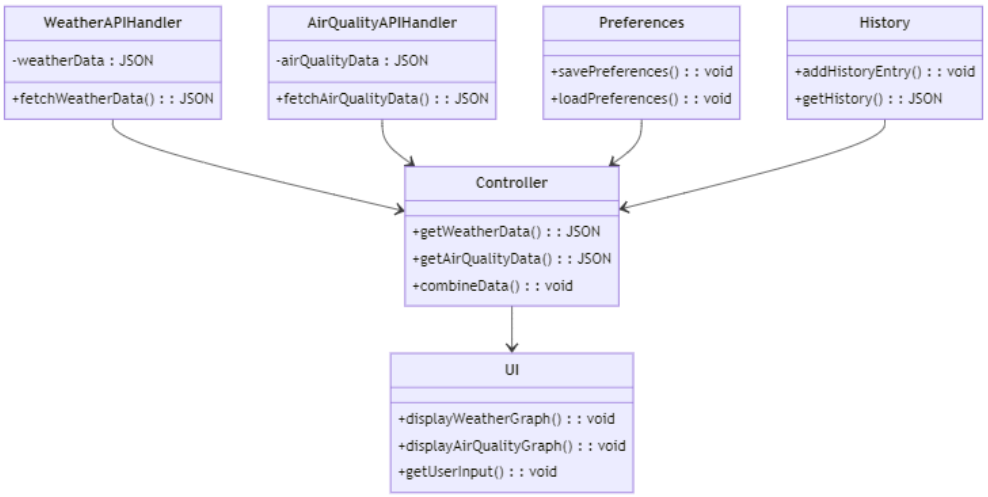


Figure 1 Event Planner's Dashboard Class Diagram

4. Design Patterns and Architecture

Several design patterns and architectural principles are employed for scalability, maintainability, and efficiency:

1. **Model-View-Controller (MVC)**:

   The **Model-View-Controller (MVC)** architecture separates the application logic, UI, and data handling to ensure modularity and maintainability. The model manages data, the view handles UI rendering, and the controller coordinates user interactions and data processing.

2. **Singleton Pattern**:

   Applied to the API handler classes, the **Singleton pattern** ensures that only one instance of the handler is created throughout the application's lifecycle. This helps manage resource usage and optimizes API calls.

3. **Factory Pattern**:

   The **Factory pattern** is employed to create objects related to API requests and data handling. It enables flexibility by allowing different types of requests (weather and air quality data) to be processed independently.

4. **Observer Pattern (Optional)**:

   This pattern could be implemented in future versions to provide real-time updates to the dashboard. For example, if live weather updates are needed, the Observer pattern could notify relevant components of changes, prompting them to refresh automatically.

5. APIs and Data

The application will retrieve data from two main APIs:

1. **Weather API**:
    - Finnish Meteorological Institute's Open Data API and Open Weather's API provide real-time weather data, such as temperature, humidity, wind speed, and precipitation forecasts.
    - **Example Endpoint**: GET https://opendata.fmi.fi/weather/forecast?location=
    - **Data Type**: JSON response with forecast details over a selected date range.
2. **Air Quality API**:
    - The Open AQ Platform API provides real-time air quality data, including PM2.5 and PM10 levels.
    - **Example Endpoint**: GET https://api.openaq.org/v1/latest?coordinates=
    - **Data Type**: JSON response with air quality metrics for the selected location.

Integration with APIs

The Event Planner's Dashboard interacts with the OpenWeatherMap API to fetch and present real-time weather data. The **WeatherApiService** class manages this integration, which includes the following steps:

1. **Data Retrieval**
   The service sends a GET request to the OpenWeatherMap API using a URL constructed with the user's city name and API key. The URL is prepared by encoding the city name and attaching additional query parameters such as units (metric system) for consistency in temperature values.
2. **Response Validation and Error Handling**
   The service checks the response code returned by the API. If an error code (e.g., 404 for city not found or 401 for an invalid API key) is encountered, the service throws an appropriate error message to notify the user. This approach ensures that invalid requests do not cause unexpected application behavior.
3. **Data Parsing and Processing**
   The JSON response from the API is parsed to extract critical weather information such as temperature, humidity, precipitation levels, and a brief weather description. The service constructs a **WeatherData** object to encapsulate this information.
4. **Controller Integration**
   The parsed weather data is returned as a **DataModel** object to the controller, which updates the application's user interface. This separation of concerns allows the controller to focus on managing UI updates while the service handles data fetching and parsing.
5. **Data Retrieval and Parsing Example**:

```
public DataModel fetchWeatherData(String cityName) {
```

```
        // Clean and encode the city name for the API request
        String encodedCity = URLEncoder.encode(cityName.trim(),
StandardCharsets.UTF_8);
        String apiUrl = String.format("%s?q=%s&units=metric&appid=%s",
BASE_URL, encodedCity, API_KEY);

        // Send the GET request and parse the JSON response
        try {
            String jsonResponse = sendGetRequest(apiUrl);
            WeatherData weatherData = parseWeatherData(jsonResponse);
            return new DataModel(List.of(weatherData));
        } catch (Exception e) {
            // Handle errors and return an empty model if needed
            System.err.println("Error fetching weather data: " +
e.getMessage());
            return new DataModel(new ArrayList<>());
        }
    }

    private String sendGetRequest(String urlString) throws IOException {
        HttpURLConnection conn = (HttpURLConnection) new
URL(urlString).openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("Accept", "application/json");

        // Validate response code and return the response as a string
        if (conn.getResponseCode() == HttpURLConnection.HTTP_OK) {
            try (BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()))) {
                return reader.lines().collect(Collectors.joining());
            }
        } else {
            throw new RuntimeException("Failed to fetch data: HTTP error
code " + conn.getResponseCode());
        }
    }
```

This structured approach to integrating the API ensures that data retrieval is reliable, errors are handled gracefully, and the user experience remains seamless.

6. Saved Preferences

The **Saved Preferences** feature allows users to store frequently used configurations, making it easy to reload previously selected locations, date ranges, and metrics without repeating the setup process.

Key Features:

1. **Preference Name**: Users can give a custom name to each saved configuration.
2. **Location and Date Range**: Saves the chosen region and date range for repeated queries.
3. **Saved Metrics**: Stores preferred weather and air quality metrics such as temperature, PM2.5, humidity, etc.

4. **Preference Actions:** Users can load, edit, or delete saved preferences through an intuitive UI.
5. **Data Storage Strategy:** Preferences are serialized into a JSON file, enabling easy retrieval and updates.
6. **Quick Overview (Optional)**: Displays a mini-summary or chart preview of the saved data.

Screen Layout:

- **Table View**: A simple table with columns for preference name, location, date range, and saved metrics.
- **Card Layout (Optional)**: Cards showing saved configurations with action buttons (Load, Edit, Delete).

7. History

The **History** section allows users to revisit previous queries, making it easy to compare past weather and air quality conditions with current data.

Key Features:

1. **Timestamp:** Records when the user last accessed a specific location or preference.
2. **Location and Date Range**: Displays previously selected regions and date ranges.
3. **Weather and Air Quality Summary**: Provides a snapshot of the weather and air quality conditions at the time.
4. **Actions**: Users can view past data or compare multiple historical entries.
5. **Optional Data**: Icons and short summaries of the past conditions for quick reference.

Screen Layout:

- **Timeline View**: A vertical timeline of past entries with timestamps, summaries, and action buttons (View Again, Compare).
- **Table View (Alternative)**: A standard table with columns for timestamp, location, weather, and air quality.

8. Tentative Approaches

In the prototype phase, the following approaches are proposed:

1. **Data Retrieval**: Asynchronous data fetching using Java's Completable Future ensures a responsive user experience by performing API calls without blocking the main thread.
2. **Visualization**: Simple line and bar graphs will be used for weather and air quality visualizations. Technologies like JavaFX or web-based libraries such as Chart.js will be explored.

3. **Customizability**: The application enables users to set custom filters, save preferences, and adjust parameters like location and date range. User preferences are stored in a local database for fast retrieval.

4. **AI Integration (Future)**: In future versions, AI-based recommendations may be implemented using historical trends to suggest optimal event dates. This could involve applying machine learning models like time series forecasting (e.g., ARIMA or LSTM) to identify patterns.

9. Self-Evaluation

In this section, we evaluate the effectiveness of our initial design and how well it has supported the development process. We follow our initial design plan, assess its potential areas of improvement, and identify any adjustments needed for the implementation.

1. **Successes:**
   - Front-End Design: We applied JavaFX for the front-end design, using SceneBuilder to streamline the creation of view components. This tool enables us to visually design the initial interface and generate corresponding FXML files, which effectively affects the development process. As a result, we successfully created four key views:
     1. About view: Provides users with an overview of the application's purpose and features.
     2. Dashboard view: Provides users with the primary interface of the application, displaying real-time weather data, calendar, and other relevant information.
     3. History view: Allows users to view the calendar of plan schedules.
     4. Save Preferences view: Enables users to configure their settings.
   - IDE choice: We utilized Apache NetBeans to develop and deploy the application.
2. **Challenges:**

   Challenge 1: FXML and Controller Synchronization
   - **Problem Description:**
     One of the challenges encountered during development was ensuring that **FXML elements were correctly synchronized with their corresponding controller fields and event handlers** in JavaFX. Mismatches between FXML-defined elements and controller fields could lead to runtime errors and unresponsive components.
   - **Planned Solution:**
     To address this issue, we plan to implement automated testing using **JUnit** and **TestFX** to verify that all FXML elements are correctly linked to their controller fields and event handlers. These tests will help ensure that updates made in SceneBuilder or FXML files do not cause unintended discrepancies in the controllers.
   - **Planned Testing Approach:**
     1. **Automated FXML Binding Verification:**
        We plan to create automated tests that load each FXML file and check that all annotated fields in the controller are correctly initialized. This will

help detect any inconsistencies between the FXML file and the controller class.

2. **Event Handler Testing:**
   Additional test cases will be designed to verify that FXML elements, such as buttons, are linked to their respective event handlers in the controller. This will prevent accidental misconfigurations during development.

- **Future Steps:**
  1. Write JUnit tests that load each FXML file and confirm that all fields are correctly initialized.
  2. Use TestFX to automate interaction tests for verifying that button clicks trigger the correct methods in the controller.

Challenge 2: Data Visualization Enhancements

- **Problem Description:**
  Another challenge faced was enhancing the **data visualizations** to be more interactive and user-friendly. The basic charts lacked features like tooltips, data filtering, and dynamic updates, making it difficult for users to explore and interpret the displayed information.
- **Planned Solution:**
  To make the data visualizations more interactive and intuitive, we have outlined several planned enhancements:
  - **Adding Interactive Tooltips:**
    We plan to implement tooltips for the weather charts to provide detailed information when users hover over specific data points. This will allow users to view temperature and humidity values quickly and accurately.
    1. **Planned Approach:**
       Utilize JavaFX's built-in Tooltip class to associate tooltips with each data point. Tooltips will dynamically display detailed information based on the hovered data point.
  - **Checkbox-Based Data Filtering:**
    To allow users to customize their view, we will add checkboxes that enable them to filter weather data based on specific metrics like temperature and humidity.
    1. **Planned Approach:**
       Modify the updateChart method to check the state of each checkbox and selectively display data series in the chart based on which options are selected.
  - **Accessibility and Interactivity Improvements:**
    We plan to introduce accessibility features to improve usability for all users, including those with disabilities. This may include adding screen reader support and enabling keyboard navigation for data exploration.
- **Future Steps:**
  - Develop a prototype of tooltips for data points in the weather chart using JavaFX's Tooltip class.
  - Implement checkbox-based filtering to allow users to control the visibility of different data series.
  - Explore options for accessibility improvements in the charts and labels.

3. **Implementation Progress:**
    - The major views (About, Dashboard, History and SavePreferences) have been implemented according to plan, aligning with the initial structure.
    - Initial data visualizations are functional, providing essential weather and air quality data, but we are still considering further enhancements.

10. Conclusion

The **Event Planner's Dashboard** is structured around providing event planners with accurate and useful data to make informed decisions about scheduling outdoor events. The separation of core components, clear user interface, and use of design patterns ensure that the application will be scalable and maintainable for future extensions. The addition of **Saved Preferences** and **History** features enhances the user experience by offering convenient data retrieval and comparison functionality.

11. Division of work

The development of the Event Planner's Dashboard involved collaborative efforts across both the front end and backend, with each team member focusing on their assigned areas:

- **Affira:**
  Affira took on a leadership role in overseeing the entire project and ensuring its progress toward mid-term submission. Affira primarily focused on the backend development, implementing core functionalities, and managing the integration between the backend and front end. Additionally, Affira played a key role in debugging issues to maintain a seamless connection between different components of the application.

- **Janika:**
  Janika contributed significantly to the backend development, with a focus on implementing the **Weather API** integration. Janika was responsible for setting up the API service, managing API key configurations, and ensuring that weather data was fetched and processed efficiently.

- **Zhiqing Feng:**
  Zhiqing led the frontend development, designing intuitive and visually appealing **UI pages** using **JavaFX** and **SceneBuilder**. Zhiqing's responsibilities included creating and styling the main views, such as the **Dashboard**, **History**, and **Preferences** screens while ensuring consistency in the user interface.

- **Naveed Ali:**
  Naveed worked alongside Zhiqing on the front-end development, focusing on implementing the visual components and interactive elements of the application. Naveed also contributed to the initial documentation, organizing the project's structure and presenting a clear overview of the application's goals and design approach.